

上机实验 3：指令动态调度性能分析

一、实验目的

- (1) 加深对指令级并行性及其开发的理解;
- (2) 加深对 Tomasulo 算法的理解;
- (3) 掌握 Tomasulo 算法在指令流出、执行、写结果各阶段对浮点作指令以及 load 和 store指令进行什么处理;
- (4) 掌握采用了 Tomasulo 算法的浮点处理部件的结构;
- (5) 掌握保留站的结构;
- (6) 给定被执行代码片段，对于具体某个时钟周期，能够写出保留站、指令状态表以及浮点寄存器状态表内容的变化情况。

二、实验平台及内容

在本次实验中，选择自己设计一个Tomasulo算法模拟器。

并实现了以下功能：

- A. 能够模拟 Tomasulo 算法的执行过程、;
- B. 支持图形交互、;
- C. 支持单步执行（一次一个时钟周期）、一次多个周期，执行到程序结束；
- D. 支持保留站、指令状态表、寄存器状态表等内容的查看；
- E. 提供程序执行后的性能统计分析；
- F. 按照 MIPS 语法，至少支持 load、store、add.d、sub.d、mul.d、div.d 操作；
- G. 要执行的程序可以直接输入运行；
- H.部件运行时间可以通过图形化界面设置。

并设计三种不同的代码组合，实现没有冲突、RAW冲突、WAR冲突的场景演示，并对场景进行分析。

三、模拟器设计思想

本次实验选择自行设计并实现一个基于Tomasulo算法的动态调度模拟器，并配备图形界面，方便观察和分析指令的动态调度、乱序执行、数据冒险消除等过程。

1. 总体分析与设计

任务分析

目标是用图形界面模拟MIPS浮点指令在Tomasulo算法下的执行过程。模拟器需能读取MIPS汇编指令，维护指令队列，动态分配保留站，跟踪寄存器状态，模拟指令的发射、执行、写回等阶段，并能直观展示各阶段状态变化。

设计思路

- **Instruction类**：表示一条MIPS指令，记录其各阶段时钟周期及状态。
- **ReservationStation类**：模拟Tomasulo算法中的保留站，记录功能部件占用情况及操作数状态。
- **ExecutionConfig类**：配置各类指令的执行周期。
- **TomasuloSimulator类**：核心模拟器，负责指令调度、保留站分配、冒险处理、时钟推进等。
- **TomasuloGUI类**：基于tkinter实现的图形界面，负责用户交互、数据显示、控制模拟流程。

2. 主要类与功能说明

Instruction类

代码：

```

class Instruction:
    def __init__(self, op, dst, src1=None, src2=None, imm=None, raw=None):
        self.op = op # 操作类型, 如add.d、load等
        self.dst = dst # 目的寄存器
        self.src1 = src1 # 源寄存器1
        self.src2 = src2 # 源寄存器2
        self.imm = imm # 立即数 (如load/store偏移)
        self.raw = raw # 原始指令文本
        self.issue = None # 发射时钟周期
        self.start_exec = None # 开始执行时钟周期
        self.end_exec = None # 结束执行时钟周期
        self.write_back = None # 写回时钟周期
        self.status = "等待" # 当前状态

```

功能：

- 表示一条MIPS指令，记录操作类型、寄存器、立即数等信息。
- 记录指令在各阶段（发射、开始执行、结束执行、写回）的时钟周期。
- 维护指令当前状态（等待、发射、执行、写回等）。

属性：

- op：操作类型（如add.d、load等）
- dst、src1、src2：目的/源寄存器
- imm：立即数
- raw：原始指令文本
- issue、start_exec、end_exec、write_back：发射周期、开始执行周期、结束执行周期、写回周期
- status：当前状态

ReservationStation类

代码：

```

class ReservationStation:
    def __init__(self, name, op_type):
        self.name = name # 保留站名称
        self.op_type = op_type # 类型 (add/mul/load)
        self.busy = False # 是否被占用
        self.op = None # 当前操作类型
        self.Vj = None # 操作数j的值
        self.Vk = None # 操作数k的值
        self.Qj = None # 操作数j的来源 (等待哪个保留站)
        self.Qk = None # 操作数k的来源
        self.A = None # 地址字段 (load/store用)
        self.instr = None # 当前指令对象
        self.remaining = 0 # 剩余执行周期
        self.Qj_ready = False
        self.Qk_ready = False

    def clear(self):
        # 清空保留站状态
        self.busy = False
        self.op = None
        self.Vj = None
        self.Vk = None
        self.Qj = None
        self.Qk = None
        self.A = None
        self.instr = None
        self.remaining = 0

```

功能：

- 模拟Tomasulo算法中的保留站，记录功能部件的占用情况。
- 跟踪操作数的就绪情况、来源、当前操作类型等。

属性：

- name：保留站名称
- op_type：功能类型（add/mul/load）
- busy：是否被占用
- op：当前操作类型
- Vj、Vk：操作数的值
- Qj、Qk：操作数的来源（等待哪个保留站）
- A：地址字段（load/store用）
- instr：当前指令对象
- remaining：剩余执行周期
- Qj_ready、Qk_ready：操作数是否就绪

解释：除Qj_ready与Qk_ready外的其余属性均是标准tomasulo算法中的设计。Qj_ready与Qk_ready则是为了防止在写回周期更新保留站的Qj,Qk后,接收到结果的保留站在同周期就开始执行操作。具体在step函数解析中说明。

ExecutionConfig类

功能：

- 配置各类指令的执行周期（如load、store、add、mul、div等）。

TomasuloSimulator类

功能：

- 实现Tomasulo算法的核心逻辑，包括指令发射、执行、写回、保留站分配、寄存器状态跟踪、数据冒险处理等。
- 支持单步、多步、全部执行等操作。
- 维护时钟周期、指令队列、保留站、寄存器、内存等状态。
- 提供状态查询、CPI计算等功能。

属性：

```
def __init__(self, exec_config):
    self.exec_config = exec_config # 执行时间配置
    self.clock = 0 # 当前时钟周期
    self.instructions = [] # 指令列表
    self.pc = 0 # 程序计数器，指向下一条待发射指令
    self.finished_instr = 0 # 已完成指令数
    # 初始化保留站
    self.res_stations = [
        ReservationStation('Add1', 'add'),
        ReservationStation('Add2', 'add'),
        ReservationStation('Add3', 'add'),
        ReservationStation('Mul1', 'mul'),
        ReservationStation('Mul2', 'mul'),
        ReservationStation('Load1', 'load'),
        ReservationStation('Load2', 'load'),
        ReservationStation('Load3', 'load'),
    ]
    # F0, F2, ..., F30
    self.reg_status = {f'F{i}': None for i in range(0, 32, 2)}
    self.reg_value = {f'F{i}': 0.0 for i in range(0, 32, 2)}
    # R寄存器，仅用于load/store寻址
    self.R_value = {f'R{i}': 0.0 for i in range(0, 32)}
    # 内存，简化为100个单元，初值1.0
    self.mem = {i: 1.0 for i in range(0, 100)}
```

- exec_config：执行周期配置
- clock：当前时钟周期

- instructions：指令列表
- pc：程序计数器
- finished_instr：已完成指令数
- res_stations：保留站列表
- reg_status：寄存器状态表（记录哪个保留站在写该寄存器）
- reg_value：寄存器值
- R_value：R寄存器值（load/store寻址用）
- mem：内存

方法：

- load_program：加载并解析MIPS程序
- parse_instruction：解析单条指令
- step：推进一个时钟周期，依次处理写回、执行、发射阶段
- step_n、run_all：多步或全部执行
- find_free_rs：查找空闲保留站
- calc_result：计算指令结果
- get_status：生成状态表字符串
- get_cpi：计算CPI（总时钟周期/指令数）

解释：

此处的step方法为模拟器最关键的设计部分，将在后面详细解释。

TomasuloGUI类

功能：

- 基于tkinter实现图形界面，负责用户输入、参数设置、状态展示、控制模拟流程。
- 支持加载程序、单步执行、多步执行、全部执行、重置等操作。
- 实时显示指令状态表、保留站状态表、寄存器状态表。
- 在所有指令执行完毕后自动弹窗显示CPI。

控件：

- 程序输入区（Text控件）
- 执行周期参数设置区（Entry控件）
- 控制按钮区（Button控件）
- 状态表显示区（Text控件）

方法：

- load_program：加载用户输入的MIPS程序，设置执行周期参数
- step、step5、run_all：控制模拟器推进
- reset：重置模拟器
- update_status：刷新状态表显示
- 在所有指令执行完毕后弹窗显示CPI

3. 关键实现思路

代码：

```

def step(self):
    # 执行一个时钟周期
    self.clock += 1
    # 1. 写回阶段
    for rs in self.res_stations:
        if rs.busy and rs.remaining == 0 and rs.instr and rs.instr.write_back is None:
            rs.instr.write_back = self.clock
            rs.instr.status = "写回"
            # 写回寄存器
            result = None
            if rs.op in ['add.d', 'sub.d', 'mul.d', 'div.d', 'load']:
                if rs.instr.dst:
                    if self.reg_status[rs.instr.dst] == rs.name:
                        self.reg_status[rs.instr.dst] = None
                    result = self.calc_result(rs)
                    self.reg_value[rs.instr.dst] = result
                else:
                    result = self.calc_result(rs)
            # Tomasulo结果广播，更新所有保留站的Qj/Qk
            for other_rs in self.res_stations:
                if other_rs.busy:
                    if other_rs.Qj == rs.name:
                        other_rs.Vj = result
                        other_rs.Qj = None
                    if other_rs.Qk == rs.name:
                        other_rs.Vk = result
                        other_rs.Qk = None
            rs.clear()
            self.finished_instr += 1
    # 2. 执行阶段
    for rs in self.res_stations:
        if rs.busy and rs.remaining > 0:
            # 操作数已就绪
            if rs.Qj_ready and rs.Qk_ready:
                if rs.instr.start_exec is None:
                    rs.instr.start_exec = self.clock
                    rs.instr.status = "执行"
                    rs.remaining -= 1
                if rs.remaining == 0:
                    rs.instr.end_exec = self.clock
    # 3. 发射阶段
    if self.pc < len(self.instructions):
        instr = self.instructions[self.pc]
        rs = self.find_free_rs(instr.op)
        if rs:
            rs.busy = True
            rs.op = instr.op
            rs.instr = instr
            if instr.op == 'load':
                # 计算地址，R寄存器寻址
                rs.A = instr.imm + self.R_value.get(instr.src1, 0.0)
                rs.remaining = self.exec_config.load
                if self.reg_status[instr.dst] is None:
                    rs.Vj = None
                    rs.Qj = None
                else:
                    rs.Qj = self.reg_status[instr.dst]
                self.reg_status[instr.dst] = rs.name
            elif instr.op == 'store':
                # 计算地址，R寄存器寻址，Vj为F寄存器的值
                rs.A = instr.imm + self.R_value.get(instr.src2, 0.0)
                rs.Vj = self.reg_value.get(instr.src1, 0.0)
                rs.remaining = self.exec_config.store
            elif instr.op in ['add.d', 'sub.d']:

```

```

rs.remaining = self.exec_config.add
# 检查源操作数是否就绪
for i, reg in enumerate([instr.src1, instr.src2]):
    if self.reg_status[reg] is None:
        if i == 0:
            rs.Vj = self.reg_value[reg]
            rs.Qj = None
        else:
            rs.Vk = self.reg_value[reg]
            rs.Qk = None
    else:
        if i == 0:
            rs.Qj = self.reg_status[reg]
        else:
            rs.Qk = self.reg_status[reg]
    self.reg_status[instr.dst] = rs.name
elif instr.op == 'mul.d':
    rs.remaining = self.exec_config.mul
    for i, reg in enumerate([instr.src1, instr.src2]):
        if self.reg_status[reg] is None:
            if i == 0:
                rs.Vj = self.reg_value[reg]
                rs.Qj = None
            else:
                rs.Vk = self.reg_value[reg]
                rs.Qk = None
        else:
            if i == 0:
                rs.Qj = self.reg_status[reg]
            else:
                rs.Qk = self.reg_status[reg]
    self.reg_status[instr.dst] = rs.name
elif instr.op == 'div.d':
    rs.remaining = self.exec_config.div
    for i, reg in enumerate([instr.src1, instr.src2]):
        if self.reg_status[reg] is None:
            if i == 0:
                rs.Vj = self.reg_value[reg]
                rs.Qj = None
            else:
                rs.Vk = self.reg_value[reg]
                rs.Qk = None
        else:
            if i == 0:
                rs.Qj = self.reg_status[reg]
            else:
                rs.Qk = self.reg_status[reg]
    self.reg_status[instr.dst] = rs.name
instr.issue = self.clock
instr.status = "发射"
self.pc += 1
for rs in self.res_stations:
    if rs.Qj is None:
        rs.Qj_ready = True
    else:
        rs.Qj_ready = False
    if rs.Qk is None:
        rs.Qk_ready = True
    else:
        rs.Qk_ready = False

def find_free_rs(self, op):
    # 查找空闲的保留站
    if op in ['add.d', 'sub.d']:
        for rs in self.res_stations:

```

```

        if rs.op_type == 'add' and not rs.busy:
            return rs
    elif op in ['mul.d', 'div.d']:
        for rs in self.res_stations:
            if rs.op_type == 'mul' and not rs.busy:
                return rs
    elif op == 'load':
        for rs in self.res_stations:
            if rs.op_type == 'load' and not rs.busy:
                return rs
    elif op == 'store':
        for rs in self.res_stations:
            if rs.op_type == 'load' and not rs.busy:
                return rs
    return None

def finished(self):
    # 判断是否所有指令都已完成
    return self.finished_instr >= len(self.instructions)

```

1. 时钟周期递增

```
self.clock += 1
```

解释：

每调用一次 `step`，表示模拟器前进一个时钟周期，所以时钟周期加1。

2. 写回阶段 (Write Back)

```

for rs in self.res_stations:
    if rs.busy and rs.remaining == 0 and rs.instr and rs.instr.write_back is None:
        rs.instr.write_back = self.clock
        rs.instr.status = "写回"
        # 写回寄存器
        result = None
        if rs.op in ['add.d', 'sub.d', 'mul.d', 'div.d', 'load']:
            if rs.instr.dst:
                if self.reg_status[rs.instr.dst] == rs.name:
                    self.reg_status[rs.instr.dst] = None
                    result = self.calc_result(rs)
                    self.reg_value[rs.instr.dst] = result
            else:
                result = self.calc_result(rs)
        # Tomasulo结果广播，更新所有保留站的Qj/Qk
        for other_rs in self.res_stations:
            if other_rs.busy:
                if other_rs.Qj == rs.name:
                    other_rs.Vj = result
                    other_rs.Qj = None
                if other_rs.Qk == rs.name:
                    other_rs.Vk = result
                    other_rs.Qk = None
        rs.clear()
    self.finished_instr += 1

```

解释：

- 遍历所有保留站，找到那些“忙碌且剩余执行周期为0且有指令且还没写回”的保留站。
- 标记该指令的写回周期为当前时钟，并将状态设为“写回”。

- 如果是需要写回寄存器的指令（如加减乘除、load），则将结果写回到目标寄存器，并清除寄存器状态表中该寄存器的占用信息。
- 计算结果后，广播到所有保留站：如果有其他保留站的Qj/Qk依赖于本保留站，则将其Vj/Vk赋值为结果，并清除Qj/Qk依赖。
- 清空本保留站，已完成指令数加1。

3. 执行阶段（Execution）

```
for rs in self.res_stations:
    if rs.busy and rs.remaining > 0:
        # 操作数已就绪
        if rs.Qj_ready and rs.Qk_ready:
            if rs.instr.start_exec is None:
                rs.instr.start_exec = self.clock
                rs.instr.status = "执行"
            rs.remaining -= 1
            if rs.remaining == 0:
                rs.instr.end_exec = self.clock
```

解释：

- 遍历所有“忙碌且剩余执行周期大于0”的保留站。
- 如果操作数都已就绪（Qj_ready、Qk_ready），则：
 - 如果是第一次进入执行阶段，记录开始执行的时钟周期，并将状态设为“执行”。
 - 执行周期减1。
 - 如果执行完毕（剩余周期为0），记录结束执行的时钟周期。

4. 发射阶段（Issue）

```
if self.pc < len(self.instructions):
    instr = self.instructions[self.pc]
    rs = self.find_free_rs(instr.op)
    if rs:
        rs.busy = True
        rs.op = instr.op
        rs.instr = instr
        # ...（不同指令类型的处理，见下）
        instr.issue = self.clock
        instr.status = "发射"
        self.pc += 1
```

解释：

- 如果还有未发射的指令（即程序计数器未到末尾），则取出下一条指令。
- 查找一个空闲的、类型匹配的保留站。
- 如果找到，设置保留站为忙碌，记录操作类型和指令对象。
- 针对不同类型的指令，做不同的初始化（如设置地址、剩余周期、操作数来源等，见下方详细说明）。
- 记录指令的发射周期和状态，并将程序计数器加1。

不同指令类型的处理：

- **load**：计算地址，设置剩余周期，目标寄存器状态表更新。
- **store**：计算地址，设置剩余周期，Vj为源寄存器的值。
- **add.d/sub.d/mul.d/div.d**：设置剩余周期，检查源操作数是否就绪，设置Vj/Vk或Qj/Qk，目标寄存器状态表更新。

5. 更新操作数就绪状态

```
for rs in self.res_stations:
    if rs.Qj is None:
        rs.Qj_ready = True
    else:
        rs.Qj_ready = False
    if rs.Qk is None:
        rs.Qk_ready = True
    else:
        rs.Qk_ready = False
```

解释：

- 遍历所有保留站，更新Qj/Qk是否就绪的标志。
- 如果Qj/Qk为None，说明操作数已就绪；否则未就绪。

6. 细节实现解释

一定是先处理写回，再处理执行，最后处理发射。如果先处理发射，再处理执行，最后处理写回，那么同一个周期可能会出现某个保留站连续进行发射执行，或者执行写回的情况。

但是这样设计出现了新问题，那就是Qj 与 Qk再写回处理完之后，便进行了更新，这将会导致等待接收结果的保留站提前执行的情况。因此，设计Qj_ready与Qk_ready变量，将两者的准备放在step函数的最后，这样就避免了执行的提前进行。

其实更规范的写法是每个step只遍历一次保留站列表，并根据保留站的state来进行相应的处理。实现中由于开始没能考虑到上述情况，没有这么写。但通过Qj_ready与Qk_ready变量的设置，同样做到了每个保留站每个step只被处理一次的效果，是完全符合tomasulo流程的。

4. 交互流程与使用体验

1. 用户输入MIPS汇编代码，点击“加载程序”。
2. 可调整各类指令的执行周期参数。
3. 点击“单步执行”、“执行5步”或“全部执行”，观察指令调度和状态变化。
4. 所有指令执行完毕后，自动弹窗显示CPI，便于性能分析。
5. 可随时点击“重置”重新开始模拟。

四、场景演示及分析

a.没有任何冲突的指令调度场景

```
load F6, 0(R2)
load F4, 4(R3)
mul.d F0, F2, F12
```

观察自己实现的模拟器与教材附带的模拟器的结果对比。

发现模拟结果完全正确。

load F6, 0(R2)
load F4, 4(R3)
mul.d F0, F2, F12

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 14

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
load F6, 0(R2)	1	2	3	4	写回
load F4, 4(R3)	2	3	4	5	写回
mul.d F0, F2, F12	3	4	13	14	写回

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	False	None	None	None	None	None	None	0
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	False	None	None	None	None	None	None	0
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	None	0.0
F4	None	1.0
F6	None	1.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

设置指令和参数，
然后点击“执行”

M[y]表示存储器中存储单元y的内容
注2:
M1=M[R[R2]]*0
M2=M[R[R3]]*4
M3=R[F2]*R[F12]

第二步：用右边的按钮，
控制指令的执行

步进 退1步 前进5个

指令状态

指令	流出	执行	写结果
L D F6, 0(R2)	1	2~3	4
L D F4, 4(R3)	2	3~4	5
MULT.D F0, F2, F12	3	4~13	14

保留站

Time	名称	Busy	Op	Vj	Vk	Qj
	Add1	No				
	Add2	No				
	Add3	No				
	Mult1	No				
	Mult2	No				

寄存器

由于没有任何冲突，简单对从第一条指令发射到写回的几个周期进行分析

第一个周期：

load F6, 0(R2)
load F4, 4(R3)
mul.d F0, F2, F12

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 1

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
load F6, 0(R2)	1	None	None	None	发射
load F4, 4(R3)	None	None	None	None	等待
mul.d F0, F2, F12	None	None	None	None	等待

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	False	None	None	None	None	None	None	0
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	True	load	None	None	None	None	0.0	2
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	None	0.0
F4	None	0.0
F6	Load1	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

检查到有空闲的load部件，*loadF6, 0(R2)*发射，目的寄存器为F6，将寄存器表中的F6更新为等待Load1保留站写入，地址A计算后写入保留站。

第二个周期：

Tomasulo模拟器

输入MIPS程序:

```
load F6, 0(R2)
load F4, 4(R3)
mul.d F0, F2, F12
```

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 2

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
load F6, 0(R2)	1	2	None	None	执行
load F4, 4(R3)	2	None	None	None	发射
mul.d F0, F2, F12	None	None	None	None	等待

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	False	None	None	None	None	None	None	0
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	True	load	None	None	None	None	0.0	1
Load2	True	load	None	None	None	None	4.0	2
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	None	0.0
F4	Load2	0.0
F6	Load1	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

检查到有空闲的load部件,loadF4,4(R3)发射,目的寄存器为F4,将寄存器表中的F4更新为等待Load2保留站写入,地址A计算后写入保留站。同时loadF6,0(R2)数据都已就绪(地址),开始执行(需两个周期)

第三个周期:

Tomasulo模拟器

输入MIPS程序:

```
load F6, 0(R2)
load F4, 4(R3)
mul.d F0, F2, F12
```

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 3

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
load F6, 0(R2)	1	2	3	None	执行
load F4, 4(R3)	2	3	None	None	执行
mul.d F0, F2, F12	3	None	None	None	发射

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	False	None	None	None	None	None	None	0
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	True	mul.d	0.0	0.0	None	None	None	10
Mul2	False	None	None	None	None	None	None	0
Load1	True	load	None	None	None	None	0.0	0
Load2	True	load	None	None	None	None	4.0	1
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	Mul1	0.0
F2	None	0.0
F4	Load2	0.0
F6	Load1	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

检查到有空闲的Mul保留站,mul.dF0,F2,F12发射,目的寄存器为F0,将寄存器表中的F0更新为等待MUL1保留站写入;而loadF4,4(R3),loadF6,0(R2)同上所述继续执行。

第四个周期:

mul.dF0,F2,F12与loadF4,4(R3)数据均就绪开始/继续执行。

loadF6,0(R2)执行了两个周期，执行完毕，进入写回周期。将寄存器状态表与保留站表更新。

b.RAW冲突的指令调度场景

```
mul.d F2, F4, F6
add.d F20, F10, F2
add.d F14, F16, F18
```

观察自己实现的模拟器与教材附带的模拟器的结果对比。
发现模拟结果完全正确。

Tomasulo模拟器

mul.d F2, F4, F6
add.d F20, F10, F2
add.d F14, F16, F18

输入MIPS程序:

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 15

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
mul.d F2, F4, F6	1	2	11	12	写回
add.d F20, F10, F2	2	13	14	15	写回
add.d F14, F16, F18	3	4	5	6	写回

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	False	None	None	None	None	None	None	0
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	False	None	None	None	None	None	None	0
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

然后点击“执行”

注2:
M1=R[F16]+R[F18]
M2=R[F4]*R[F6]
M3=R[F10]+M2

第二步：用右边的按钮，控制指令的执行

步进 退1步 前进5个周期

指令状态

指令	流出	执行	写结果
MULT.D F2, F4, F6	1	2~11	12
ADD.D F20, F10, F2	2	13~14	15
ADD.D F14, F16, F18	3	4~5	6

保留站

Time	名称	Busy	Op	Vj	Vk	Qi	Qk
------	----	------	----	----	----	----	----

RAW场景分析：在mul.dF2,F4,F6指令中，目的寄存器为F2，而在第二条指令add.dF20,F10,F2中，源操作数寄存器有F2,若第二条指令在第一条指令写入之前就读取了F2，就发生了RAW冲突。

现在分析tomasulo是如何避免这个冲突的。

在周期2, add.dF20,F10,F2指令被发射到add1保留站; tomasulo模拟器检查指令的源操作数，发现需要等待Mul1的结果（第一条指令在周期1被发射到了Mul1并更新寄存器状态表）,因此将add1保留站的Qk标记为Mul1，需要等待mul1保留站的结果;

Tomasulo模拟器

输入MIPS程序:

```
mul.d F2, F4, F6
add.d F20, F10, F2
add.d F14, F16, F18
```

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 2

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
mul.d F2, F4, F6	1	2	None	None	执行
add.d F20, F10, F2	2	None	None	None	发射
add.d F14, F16, F18	None	None	None	None	等待

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	True	add.d	0.0	None	None	Mul1	None	2
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	True	mul.d	0.0	0.0	None	None	None	9
Mul2	False	None	None	None	None	None	None	0
Load1	False	None	None	None	None	None	None	0
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	Mul1	0.0
F4	None	0.0
F6	None	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

在后续的数个周期里, *add.d F20, F10, F2* 的指令状态被停滞了, 没有推进下去, 因为在等待Mul1的结果。Mul部件执行时间被设定为10个周期; 以周期6状态举例:

Tomasulo模拟器

输入MIPS程序:

```
mul.d F2, F4, F6
add.d F20, F10, F2
add.d F14, F16, F18
```

Load/Store时间: 2 Mul时间: 10 Div时间: 40 Add/Sub时间: 2

加载程序 单步执行 执行5步 全部执行 重置

时钟周期: 6

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
mul.d F2, F4, F6	1	2	None	None	执行
add.d F20, F10, F2	2	None	None	None	发射
add.d F14, F16, F18	3	4	5	6	写回

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	True	add.d	0.0	None	None	Mul1	None	2
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	True	mul.d	0.0	0.0	None	None	None	5
Mul2	False	None	None	None	None	None	None	0
Load1	False	None	None	None	None	None	None	0
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	Mul1	0.0
F4	None	0.0
F6	None	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

由于Mul部件设定执行时间为10, 直到周期12, *add.d F20, F10, F2* 写回, 更新保留站的Qj, Qk

Tomasulo模拟器

mul.d F2, F4, F6

add.d F20, F10, F2

add.d F14, F16, F18

输入MIPS程序:

Load/Store时间: 2

Mul时间: 10

Div时间: 40

Add/Sub时间: 2

加载程序

单步执行

执行5步

全部执行

重置

时钟周期: 12

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
mul.d F2, F4, F6	1	2	11	12	写回
add.d F20, F10, F2	2	None	None	None	发射
add.d F14, F16, F18	3	4	5	6	写回

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	True	add.d	0.0	0.0	None	None	None	2
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	False	None	None	None	None	None	None	0
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	None	0.0
F4	None	0.0
F6	None	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

下一个周期13, *add.d*F20, F10, F2才能开始执行。

Tomasulo模拟器

mul.d F2, F4, F6

add.d F20, F10, F2

add.d F14, F16, F18

输入MIPS程序:

Load/Store时间: 2

Mul时间: 10

Div时间: 40

Add/Sub时间: 2

加载程序

单步执行

执行5步

全部执行

重置

时钟周期: 13

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
mul.d F2, F4, F6	1	2	11	12	写回
add.d F20, F10, F2	2	13	None	None	执行
add.d F14, F16, F18	3	4	5	6	写回

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	True	add.d	0.0	0.0	None	None	None	1
Add2	False	None	None	None	None	None	None	0
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	False	None	None	None	None	None	None	0
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

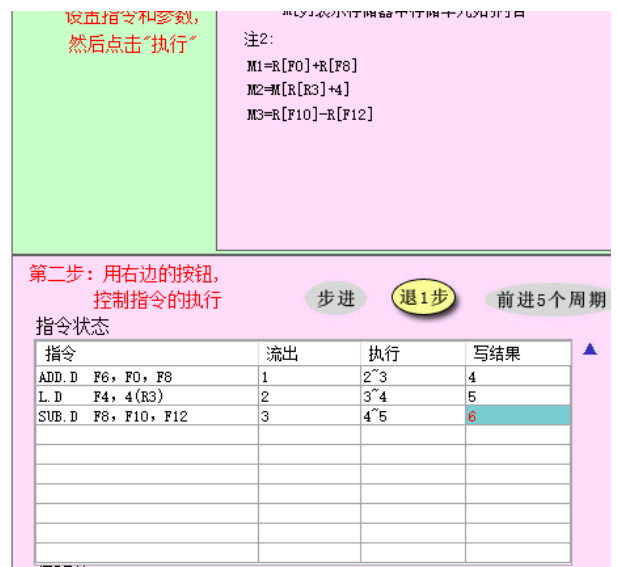
寄存器	Qi	值
F0	None	0.0
F2	None	0.0
F4	None	0.0
F6	None	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

成功避免了RAW冲突。

c.WAR冲突的指令调度场景

```
add.d F6, F0, F8
load F4, 4(R3)
sub.d F8, F10, F12s
```

发现模拟结果完全正确。



现在分析tomasulo是如何避免这个冲突的。

Tomasulo模拟器

add.d F6, F0, F8
 load F4, 4(R3)
 sub.d F8, F10, F12

输入MIPS程序:

Load/Store时间: Mul时间: Div时间: Add/Sub时间:

加载程序

时钟周期:

[指令状态表]		发射	开始执行	结束执行	写回	状态
指令						
add.d F6, F0, F8	1	None	None	None	发射	
load F4, 4(R3)	None	None	None	None	等待	
sub.d F8, F10, F12	None	None	None	None	等待	

[保留站状态表]		Vj	Vk	Qj	Qk	A	剩余
名称 Busy Op							
Add1 True add.d	0.0	0.0	None	None	None	2	
Add2 False None	None	None	None	None	None	0	
Add3 False None	None	None	None	None	None	0	
Mul1 False None	None	None	None	None	None	0	
Mul2 False None	None	None	None	None	None	0	
Load1 False None	None	None	None	None	None	0	
Load2 False None	None	None	None	None	None	0	
Load3 False None	None	None	None	None	None	0	

[寄存器状态表]		值
寄存器 Qi		
F0	None	0.0
F2	None	0.0
F4	None	0.0
F6	Add1	0.0
F8	None	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

在周期3，sub.d命令发射时，寄存器状态表更新，记录F8等待保留站Add2的结果

Tomasulo模拟器

add.d F6, F0, F8
load F4, 4(R3)
sub.d F8, F10, F12

输入MIPS程序:

Load/Store时间: 2Mul时间: 10Div时间: 40Add/Sub时间: 2

加载程序单步执行执行5步全部执行重置

时钟周期: 3

[指令状态表]

指令	发射	开始执行	结束执行	写回	状态
add.d F6, F0, F8	1	2	3	None	执行
load F4, 4(R3)	2	3	None	None	执行
sub.d F8, F10, F12	3	None	None	None	发射

[保留站状态表]

名称	Busy	Op	Vj	Vk	Qj	Qk	A	剩余
Add1	True	add.d	0.0	0.0	None	None	None	0
Add2	True	sub.d	0.0	0.0	None	None	None	2
Add3	False	None	None	None	None	None	None	0
Mul1	False	None	None	None	None	None	None	0
Mul2	False	None	None	None	None	None	None	0
Load1	True	load	None	None	None	None	4.0	1
Load2	False	None	None	None	None	None	None	0
Load3	False	None	None	None	None	None	None	0

[寄存器状态表]

寄存器	Qi	值
F0	None	0.0
F2	None	0.0
F4	Load1	0.0
F6	Add1	0.0
F8	Add2	0.0
F10	None	0.0
F12	None	0.0
F14	None	0.0
F16	None	0.0

这两步实际上进行了寄存器换名并对操作数进行了缓冲，从而消除了WAR冲突。

即

```
add.d F6, F0, F8
load F4, 4(R3)
sub.d **Add2**, F10, F12
```

并最终将add2的结果写入F8。

五、实验感悟

通过本次Tomasulo算法动态调度模拟器的设计与实现，我对指令级并行性开发等体系结构核心技术有了更深刻的理解。Tomasulo算法的实现考虑RAW、WAR、WAW等多种数据冒险，设计了动态分配保留站、跟踪寄存器状态、实现结果转发和寄存器重命名等机制。所有这些复杂的调度和冒险消除逻辑，在本次实验中都需要通过代码一一实现和调试，这让我对理论知识有了直观的体会。

此外，实验中通过图形界面实时观察指令状态表、保留站状态表、寄存器状态表的变化，能够清晰地看到各种冒险是如何被动态消除的，进一步体会到动态调度技术对处理器性能提升的关键作用。

总之，本次实验不仅锻炼了我的编程能力，让我对动态调度、数据冒险消除等关键技术有了系统、深入的理解。理论与实践的结合，为后续学习和研究相关内容打下了坚实的基础。

六、附录

```
python tm.py
```

运行模拟器。

tm.py:


```

import tkinter as tk
from tkinter import ttk, messagebox

# 指令对象，表示一条MIPS指令及其状态
class Instruction:
    def __init__(self, op, dst, src1=None, src2=None, imm=None, raw=None):
        self.op = op # 操作类型，如add.d、load等
        self.dst = dst # 目的寄存器
        self.src1 = src1 # 源寄存器1
        self.src2 = src2 # 源寄存器2
        self.imm = imm # 立即数（如load/store偏移）
        self.raw = raw # 原始指令文本
        self.issue = None # 发射时钟周期
        self.start_exec = None # 开始执行时钟周期
        self.end_exec = None # 结束执行时钟周期
        self.write_back = None # 写回时钟周期
        self.status = "等待" # 当前状态

```

保留站对象，表示一个功能部件的占用情况

```

class ReservationStation:
    def __init__(self, name, op_type):
        self.name = name # 保留站名称
        self.op_type = op_type # 类型（add/mul/load）
        self.busy = False # 是否被占用
        self.op = None # 当前操作类型
        self.Vj = None # 操作数j的值
        self.Vk = None # 操作数k的值
        self.Qj = None # 操作数j的来源（等待哪个保留站）
        self.Qk = None # 操作数k的来源
        self.A = None # 地址字段（load/store用）
        self.instr = None # 当前指令对象
        self.remaining = 0 # 剩余执行周期
        self.Qj_ready = False
        self.Qk_ready = False

    def clear(self):
        # 清空保留站状态
        self.busy = False
        self.op = None
        self.Vj = None
        self.Vk = None
        self.Qj = None
        self.Qk = None
        self.A = None
        self.instr = None
        self.remaining = 0

```

执行时间配置对象

```

class ExecutionConfig:
    def __init__(self, load=2, store=2, add=2, mul=10, div=40):
        self.load = load # load指令执行周期
        self.store = store # store指令执行周期
        self.add = add # add/sub指令执行周期
        self.mul = mul # mul指令执行周期
        self.div = div # div指令执行周期

```

Tomasulo算法模拟器核心

```

class TomasuloSimulator:
    def __init__(self, exec_config):
        self.exec_config = exec_config # 执行时间配置
        self.clock = 0 # 当前时钟周期
        self.instructions = [] # 指令列表
        self.pc = 0 # 程序计数器，指向下一条待发射指令
        self.finished_instr = 0 # 已完成指令数

```

```

# 初始化保留站
self.res_stations = [
    ReservationStation('Add1', 'add'),
    ReservationStation('Add2', 'add'),
    ReservationStation('Add3', 'add'),
    ReservationStation('Mul1', 'mul'),
    ReservationStation('Mul2', 'mul'),
    ReservationStation('Load1', 'load'),
    ReservationStation('Load2', 'load'),
    ReservationStation('Load3', 'load'),
]

# F0, F2, ..., F30
self.reg_status = {f'F{i}': None for i in range(0, 32, 2)}
self.reg_value = {f'F{i}': 0.0 for i in range(0, 32, 2)}
# R寄存器, 仅用于load/store寻址
self.R_value = {f'R{i}': 0.0 for i in range(0, 32)}
# 内存, 简化为100个单元, 初值1.0
self.mem = {i: 1.0 for i in range(0, 100)}

def reset(self):
    # 重置模拟器状态
    self.clock = 0
    self.pc = 0
    self.finished_instr = 0
    for rs in self.res_stations:
        rs.clear()
    for k in self.reg_status:
        self.reg_status[k] = None
    for k in self.reg_value:
        self.reg_value[k] = 0.0
    self.mem = {i: 1.0 for i in range(0, 100)}
    for instr in self.instructions:
        instr.issue = instr.start_exec = instr.end_exec = instr.write_back = None
        instr.status = "等待"

def load_program(self, program):
    # 加载并解析MIPS程序
    self.instructions = []
    for line in program.strip().split('\n'):
        line = line.strip()
        if not line or line.startswith('#'):
            continue
        instr = self.parse_instruction(line)
        if instr:
            self.instructions.append(instr)
    self.reset()

def parse_instruction(self, line):
    # 解析一条MIPS指令, 支持load/store/add.d/sub.d/mul.d/div.d
    tokens = line.replace(',', ' ').replace('(', ' ').replace(')', ' ').split()
    if not tokens:
        return None
    op = tokens[0].lower()
    if op == 'load':
        # load F2, 0(R4)
        dst = tokens[1]
        addr_reg = tokens[3]
        if not (dst.startswith('F') and dst[1:].isdigit() and int(dst[1:]) % 2 == 0):
            return None # 只允许F0, F2, ..., F30
        return Instruction('load', dst, addr_reg, imm=int(tokens[2]), raw=line)
    elif op == 'store':
        # store F2, 0(R4)
        src = tokens[1]
        addr_reg = tokens[3]
        if not (src.startswith('F') and src[1:].isdigit() and int(src[1:]) % 2 == 0):

```

```

        return None # 只允许F0, F2, ..., F30
    return Instruction('store', src, addr_reg, imm=int(tokens[2]), raw=line)
elif op in ['add.d', 'sub.d', 'mul.d', 'div.d']:
    # add.d F2, F4, F6
    dst, src1, src2 = tokens[1], tokens[2], tokens[3]
    # 检查寄存器编号
    for reg in [dst, src1, src2]:
        if not (reg.startswith('F') and reg[1:].isdigit() and int(reg[1:]) % 2 == 0):
            return None
    return Instruction(op, dst, src1, src2, raw=line)
else:
    return None

def step(self):
    # 执行一个时钟周期
    self.clock += 1
    # 1. 写回阶段
    for rs in self.res_stations:
        if rs.busy and rs.remaining == 0 and rs.instr and rs.instr.write_back is None:
            rs.instr.write_back = self.clock
            rs.instr.status = "写回"
            # 写回寄存器
            result = None
            if rs.op in ['add.d', 'sub.d', 'mul.d', 'div.d', 'load']:
                if rs.instr.dst:
                    if self.reg_status[rs.instr.dst] == rs.name:
                        self.reg_status[rs.instr.dst] = None
                    result = self.calc_result(rs)
                    self.reg_value[rs.instr.dst] = result
            else:
                result = self.calc_result(rs)
            # Tomasulo结果广播, 更新所有保留站的Qj/Qk
            for other_rs in self.res_stations:
                if other_rs.busy:
                    if other_rs.Qj == rs.name:
                        other_rs.Vj = result
                        other_rs.Qj = None
                    if other_rs.Qk == rs.name:
                        other_rs.Vk = result
                        other_rs.Qk = None
            rs.clear()
            self.finished_instr += 1
    # 2. 执行阶段
    for rs in self.res_stations:
        if rs.busy and rs.remaining > 0:
            # 操作数已就绪
            if rs.Qj_ready and rs.Qk_ready:
                if rs.instr.start_exec is None:
                    rs.instr.start_exec = self.clock
                    rs.instr.status = "执行"
                rs.remaining -= 1
                if rs.remaining == 0:
                    rs.instr.end_exec = self.clock
    # 3. 发射阶段
    if self.pc < len(self.instructions):
        instr = self.instructions[self.pc]
        rs = self.find_free_rs(instr.op)
        if rs:
            rs.busy = True
            rs.op = instr.op
            rs.instr = instr
            if instr.op == 'load':
                # 计算地址, R寄存器寻址
                rs.A = instr.imm + self.R_value.get(instr.src1, 0.0)
                rs.remaining = self.exec_config.load

```

```

        if self.reg_status[instr.dst] is None:
            rs.Vj = None
            rs.Qj = None
        else:
            rs.Qj = self.reg_status[instr.dst]
            self.reg_status[instr.dst] = rs.name
    elif instr.op == 'store':
        # 计算地址, R寄存器寻址, Vj为F寄存器的值
        rs.A = instr.imm + self.R_value.get(instr.src2, 0.0)
        rs.Vj = self.reg_value.get(instr.src1, 0.0)
        rs.remaining = self.exec_config.store
    elif instr.op in ['add.d', 'sub.d']:
        rs.remaining = self.exec_config.add
        # 检查源操作数是否就绪
        for i, reg in enumerate([instr.src1, instr.src2]):
            if self.reg_status[reg] is None:
                if i == 0:
                    rs.Vj = self.reg_value[reg]
                    rs.Qj = None
                else:
                    rs.Vk = self.reg_value[reg]
                    rs.Qk = None
            else:
                if i == 0:
                    rs.Qj = self.reg_status[reg]
                else:
                    rs.Qk = self.reg_status[reg]
        self.reg_status[instr.dst] = rs.name
    elif instr.op == 'mul.d':
        rs.remaining = self.exec_config.mul
        for i, reg in enumerate([instr.src1, instr.src2]):
            if self.reg_status[reg] is None:
                if i == 0:
                    rs.Vj = self.reg_value[reg]
                    rs.Qj = None
                else:
                    rs.Vk = self.reg_value[reg]
                    rs.Qk = None
            else:
                if i == 0:
                    rs.Qj = self.reg_status[reg]
                else:
                    rs.Qk = self.reg_status[reg]
        self.reg_status[instr.dst] = rs.name
    elif instr.op == 'div.d':
        rs.remaining = self.exec_config.div
        for i, reg in enumerate([instr.src1, instr.src2]):
            if self.reg_status[reg] is None:
                if i == 0:
                    rs.Vj = self.reg_value[reg]
                    rs.Qj = None
                else:
                    rs.Vk = self.reg_value[reg]
                    rs.Qk = None
            else:
                if i == 0:
                    rs.Qj = self.reg_status[reg]
                else:
                    rs.Qk = self.reg_status[reg]
        self.reg_status[instr.dst] = rs.name
    instr.issue = self.clock
    instr.status = "发射"
    self.pc += 1
for rs in self.res_stations:
    if rs.Qj is None:

```

```

        rs.Qj_ready = True
    else:
        rs.Qj_ready = False
    if rs.Qk is None:
        rs.Qk_ready = True
    else:
        rs.Qk_ready = False

def step_n(self, n):
    # 连续执行n个时钟周期
    for _ in range(n):
        if self.finished_instr >= len(self.instructions):
            break
        self.step()

def run_all(self):
    # 一直执行到所有指令完成
    while self.finished_instr < len(self.instructions):
        self.step()

def find_free_rs(self, op):
    # 查找空闲的保留站
    if op in ['add.d', 'sub.d']:
        for rs in self.res_stations:
            if rs.op_type == 'add' and not rs.busy:
                return rs
    elif op in ['mul.d', 'div.d']:
        for rs in self.res_stations:
            if rs.op_type == 'mul' and not rs.busy:
                return rs
    elif op == 'load':
        for rs in self.res_stations:
            if rs.op_type == 'load' and not rs.busy:
                return rs
    elif op == 'store':
        for rs in self.res_stations:
            if rs.op_type == 'load' and not rs.busy:
                return rs
    return None

def calc_result(self, rs):
    # 计算指令结果
    if rs.op == 'add.d':
        return rs.Vj + rs.Vk
    elif rs.op == 'sub.d':
        return rs.Vj - rs.Vk
    elif rs.op == 'mul.d':
        return rs.Vj * rs.Vk
    elif rs.op == 'div.d':
        return rs.Vj / rs.Vk if rs.Vk != 0 else 0.0
    elif rs.op == 'load':
        return self.mem.get(int(rs.A), 0.0)
    return 0.0

def finished(self):
    # 判断是否所有指令都已完成
    return self.finished_instr >= len(self.instructions)

def get_status(self):
    # 生成状态表字符串, 包含指令状态、保留站状态、寄存器状态
    s = f"时钟周期: {self.clock}\n\n[指令状态表]\n"
    s += f"{'指令':<30}{'发射':<6}{'开始执行':<8}{'结束执行':<8}{'写回':<6}{'状态':<6}\n"
    for instr in self.instructions:
        s += f"{instr.raw:<30}{str(instr.issue):<6}{str(instr.start_exec):<8}{str(instr.end_exec):<8}{str(instr.write_back):<6}{i
    # 保留站状态表

```

```

s += "\n[保留站状态表]\n"
s += f"{'名称':<6}{'Busy':<6}{'Op':<8}{'Vj':<8}{'Vk':<8}{'Qj':<8}{'Qk':<8}{'A':<8}{'剩余':<6}\n"
for rs in self.res_stations:
    s += f"{'rs.name':<6}{str(rs.busy):<6}{str(rs.op):<8}{str(rs.vj):<8}{str(rs.vk):<8}{str(rs.qj):<8}{str(rs.qk):<8}{str(rs.A)}\n"
# 寄存器状态表
s += "\n[寄存器状态表]\n"
s += f"{'寄存器':<8}{'Qi':<8}{'值':<8}\n"
for k in sorted(self.reg_status.keys(), key=lambda x: int(x[1:])):
    s += f"{'k':<8}{str(self.reg_status[k]):<8}{str(self.reg_value[k]):<8}\n"
return s

def get_cpi(self):
    if len(self.instructions) == 0:
        return 0.0
    return self.clock / len(self.instructions)

# 图形界面类，负责tkinter窗口和交互
class TomasuloGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Tomasulo模拟器")
        self.exec_config = ExecutionConfig() # 执行时间配置
        self.sim = TomasuloSimulator(self.exec_config) # 模拟器实例
        self.create_widgets() # 创建界面控件

    def create_widgets(self):
        # 程序输入区
        frame1 = tk.Frame(self.root)
        frame1.pack(fill=tk.X)
        tk.Label(frame1, text="输入MIPS程序:").pack(side=tk.LEFT)
        self.program_text = tk.Text(frame1, height=8, width=60)
        self.program_text.pack(side=tk.LEFT)
        # 执行时间调整区
        frame2 = tk.Frame(self.root)
        frame2.pack(fill=tk.X)
        tk.Label(frame2, text="Load/Store时间:").pack(side=tk.LEFT)
        self.load_time = tk.IntVar(value=2)
        tk.Entry(frame2, textvariable=self.load_time, width=3).pack(side=tk.LEFT)
        tk.Label(frame2, text="Mul时间:").pack(side=tk.LEFT)
        self.mul_time = tk.IntVar(value=10)
        tk.Entry(frame2, textvariable=self.mul_time, width=3).pack(side=tk.LEFT)
        tk.Label(frame2, text="Div时间:").pack(side=tk.LEFT)
        self.div_time = tk.IntVar(value=40)
        tk.Entry(frame2, textvariable=self.div_time, width=3).pack(side=tk.LEFT)
        tk.Label(frame2, text="Add/Sub时间:").pack(side=tk.LEFT)
        self.add_time = tk.IntVar(value=2)
        tk.Entry(frame2, textvariable=self.add_time, width=3).pack(side=tk.LEFT)
        # 控制按钮区
        frame3 = tk.Frame(self.root)
        frame3.pack(fill=tk.X)
        tk.Button(frame3, text="加载程序", command=self.load_program).pack(side=tk.LEFT)
        tk.Button(frame3, text="单步执行", command=self.step).pack(side=tk.LEFT)
        tk.Button(frame3, text="执行5步", command=self.step5).pack(side=tk.LEFT)
        tk.Button(frame3, text="全部执行", command=self.run_all).pack(side=tk.LEFT)
        tk.Button(frame3, text="重置", command=self.reset).pack(side=tk.LEFT)
        # 状态表显示区
        frame4 = tk.Frame(self.root)
        frame4.pack(fill=tk.BOTH, expand=True)
        self.status_table = tk.Text(frame4, height=30, width=100)
        self.status_table.pack(fill=tk.BOTH, expand=True)

    def load_program(self):
        # 加载用户输入的MIPS程序
        program = self.program_text.get(1.0, tk.END)
        # 设置执行时间参数

```

```

self.exec_config.load = self.load_time.get()
self.exec_config.store = self.load_time.get()
self.exec_config.mul = self.mul_time.get()
self.exec_config.div = self.div_time.get()
self.exec_config.add = self.add_time.get()
self.sim.load_program(program)
self.update_status()

def step(self):
    # 单步执行
    if self.sim.finished():
        messagebox.showinfo("提示", "程序已执行完毕! \nCPI: %.2f" % self.sim.get_cpi())
        return
    self.sim.step()
    self.update_status()
    if self.sim.finished():
        messagebox.showinfo("提示", "程序已执行完毕! \nCPI: %.2f" % self.sim.get_cpi())

def step5(self):
    # 连续执行5步
    if self.sim.finished():
        messagebox.showinfo("提示", "程序已执行完毕! \nCPI: %.2f" % self.sim.get_cpi())
        return
    self.sim.step_n(5)
    self.update_status()
    if self.sim.finished():
        messagebox.showinfo("提示", "程序已执行完毕! \nCPI: %.2f" % self.sim.get_cpi())

def run_all(self):
    # 一直执行到结束
    if self.sim.finished():
        messagebox.showinfo("提示", "程序已执行完毕! \nCPI: %.2f" % self.sim.get_cpi())
        return
    self.sim.run_all()
    self.update_status()
    if self.sim.finished():
        messagebox.showinfo("提示", "程序已执行完毕! \nCPI: %.2f" % self.sim.get_cpi())

def reset(self):
    # 重置模拟器
    self.sim.reset()
    self.update_status()

def update_status(self):
    # 刷新状态表显示
    self.status_table.delete(1.0, tk.END)
    self.status_table.insert(tk.END, self.sim.get_status())

if __name__ == '__main__':
    # 启动主窗口
    root = tk.Tk()
    app = TomasuloGUI(root)
    root.mainloop()

```